Parallel and Distributed Programming
OpenMP for GPU

Kenjiro Taura

2026/11/02

OpenMP for GPU

- recent OpenMP supports offloading to GPU (`target` directive)
- official home page: http://openmp.org/
- specification: http://openmp.org/wp/openmp-specifications/
- latest version is 5.0 (https://www.openmp.org/spec-html/5.0/openmp.html)
- section numbers below refer to those in OpenMP spec 5.0

Compiling OpenMP programs for GPUs

- LLVM (`clang/clang++`) : compile with `-fopenmp --offload-arch=sm_90`

```
$ clang++ -Wall -fopenmp --offload-arch=sm_90 program.cc
```

- NVIDIA HPC SDK (`nvc/nvc++`) : compile with `-mp=gpu`

```
$ nvc -Wall -mp=gpu program.c
```

Directives overview

1. move work to device
   - NEW `target` : moves the execution to GPU
2. parallelize
   - NEW `teams` and `distribute`
     - `teams` : creates a number of teams executing the same statement (≈ `parallel` pragma)
     - `distribute` : distribute iterations of a for loop among teams (≈ `for` pragma)
   - `parallel` and `for`
     - `parallel` : creates a number of threads executing the same statement in a team
     - `for` : distribute iterations of a for loop among threads in a team
   - think of `teams` + `distribute` as another layer outside `parallel` + `for`
3. move (or sync) data
   - NEW `map` : move/sync data between CPU and GPU

Implementation note

- while not specified anywhere in the spec (and there are cases they behave differently to below), you can think of
  - a team $\sim$ a thread block
  - a thread $\sim$ a CUDA thread
- it at least helps you understand why things look so redundant …

Frequently-used combined idioms

- execute a loop on the whole GPU

```
#pragma omp target teams distribute parallel for
for (int i = start; i < end; i += incr) {
  S
}
```

- `teams` + `distribute` to the outer loop and `parallel` + `for` to the inner loop

```
#pragma omp target teams distribute
for (int i = start; i < end; i += incr) {
#pragma omp parallel for
  for (int j = start'; j < end'; j += incr') {
    S
  }
}
```

- similar to launching a kernel doing S, but you don't adjust thread count

Data sharing between CPU and GPU (`target`)

- Thanks to CPU-GPU shared memory, it's largely automatic on Miyabi (GH200)
- The following "just works" (prints `a[0] = 6`)

```
int main() {
  int a[1] = { 5 };
#pragma omp target
  a[0]++;
  printf("a[0] = %d\n", a[0]);
}
```

- Equally works when `a` is global or heap-allocated (malloc/new/mmap)
- The only exception is local/global scalar variables

Local/global scalar variables

- The following prints `s = 5`, not `s = 6`

```
int main() {
  int s = 5;
#pragma omp target
  s++;
  printf("s = %d\n", s);
}
```

- GPU automatically reads values set by CPU, *but not vice versa*
- adding `map(tofrom: v)` or `defaultmap(tofrom: scalar)` fixes that

```
int main() {
  int s = 5;
#pragma omp target map(tofrom: s) // or defaultmap(tofrom: scalar)
  s++;
  printf("s = %d\n", s);
}
```

OpenMP data mapping in general

```
#pragma omp target map(to:u) map(from:v)
   S
```

- u, v : a variable name (scalar or array) or a range expression (e.g., `a[p:q]`)
  - `to:` send u *to* GPU *before* S
  - `from:` send v *from* GPU *after* S
- you can combine `to:` and `from:` into `tofrom:`
- somewhat "declarative" way of understanding this
  - `to:` u becomes valid ("mapped") on GPU during S
  - `from:` v becomes valid on CPU after S
- again, they are largely for GPUs with separate memory (not logically shared with CPU)

Synchronization primitives

- There are primitives available on CPU but not on GPU

`#pragma omp ...`	supported?
`atomic`	yes
`reduction`	yes
`barrier`	only within a team
`critical`	no (*)

- (*) clang++ compiles it but silently returns a wrong answer